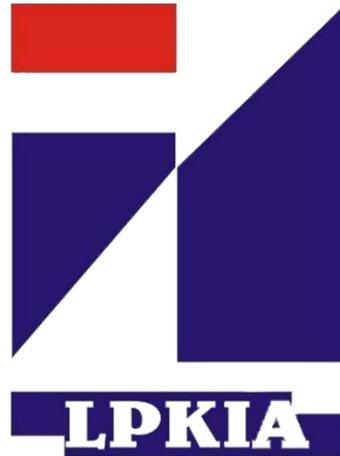


LAPORAN
APLIKASI KAMUS DIGITAL MULTILINGUAL



Mata kuliah: Pengantar Basis Data
Dosen pengampu: Dayani Vera Versanika, S.T. , M.Kom.

Disusun Oleh:

Renjiro Mulyawisesa	250414038
Agna Aryaputra Rahman	250414019
Zaki Yusron Hasyimmi	250414010
Rasya Hilalun Nadhir	250414040

TEKNIK INFORMATIKA
INSTITUT DIGITAL EKONOMI LPKIA
2026

LEMBAR PENGESAHAN

Laporan proyek ini berjudul Aplikasi Kamus Digital Multilingual Berbasis Trie, Hash Table, dan Doubly Linked List yang disusun oleh kelompok 9 (Sembilan) mahasiswa S1 Teknik Informatika Digital Ekonomi LPKIA yang beranggotakan:

- | | |
|--------------------------|-----------|
| 1. Zaki Yusron Hasyimmi | 250414010 |
| 2. Renjiro Mulyawisesa | 250414038 |
| 3. Rasya Hilalun Nadhir | 250414040 |
| 4. Agna Aryaputra Rahman | 250414019 |

Laporan ini telah diperiksa dan disetujui sebagai salah satu syarat penilaian mata kuliah Struktur Data.

Disetujui,
Dosen Pengampun

Dayani Vera Versanika, S.T. , M.Kom.

KATA PENGANTAR

Puji dan syukur kami panjatkan ke hadirat Tuhan Yang Maha Esa atas rahmat dan karunia-Nya sehingga kami dapat menyelesaikan laporan proyek ini dengan baik. Laporan berjudul Aplikasi Kamus Digital Multilingual Berbasis Trie, Hash Table, dan Doubly Linked List ini disusun sebagai salah satu pemenuhan tugas akhir mata kuliah Struktur Data.

Aplikasi yang kami kembangkan merupakan kamus digital yang mendukung tiga bahasa — Bahasa Indonesia, Bahasa Inggris, dan Bahasa Sunda — dengan mengedepankan efisiensi pencarian melalui penerapan struktur data tingkat lanjut. Pemilihan Trie untuk fitur autocomplete, Hash Table untuk akses data konstan, dan Doubly Linked List untuk manajemen histori pencarian merupakan hasil kajian mendalam terhadap kebutuhan dan karakteristik data linguistik yang kami miliki.

Kami menyadari bahwa laporan ini masih jauh dari sempurna. Oleh karena itu, kami sangat mengharapkan masukan, saran, dan kritik yang membangun dari dosen pengampu dan seluruh pembaca. Semoga laporan ini dapat memberikan manfaat, baik sebagai bahan referensi akademis maupun sebagai inspirasi pengembangan aplikasi kamus digital di masa mendatang.

Akhir kata, kami mengucapkan terima kasih sebesar-besarnya kepada dosen pengampu mata kuliah Struktur Data yang telah membimbing dan mengarahkan kami sepanjang proses pengerjaan proyek ini, serta kepada seluruh pihak yang telah memberikan dukungan moral dan teknis.

Bandung, 5 Juli 2026

Tim Penyusun

DAFTAR ISI

LEMBAR PENGESAHAN	i
KATA PENGANTAR	i
DAFTAR ISI.....	ii
DAFTAR GAMBAR	iv
DAFTAR TABEL	v
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	2
1.3 Tujuan	3
1.4 Manfaat	3
1.5 Ruang Lingkup.....	4
BAB II TINJAUAN PUSTAKA.....	5
2.1 Dasar Teori Struktur Data	5
2.1.1 Trie (Prefix Tree)	5
2.1.2 Hash Table	6
2.1.3 Doubly Linked List	6
2.2 Algoritma Terkait.....	7
2.2.1 Polynomial Rolling Hash	7
2.2.2 Depth-First Search (DFS) pada Trie	8
2.2.3 Levenshtein Distance (Saran Koreksi Ejaan).....	8
2.3 Analisis Kompleksitas.....	8
2.4 Tools/Teknologi	9
BAB III ANALISA DAN PERANCANGAN	10
3.1 Analisis Kebutuhan	10
3.1.1 Kebutuhan Fungsional	10
3.1.2 Kebutuhan Non-Fungsional	11
3.2 Perancangan Arsitektur	11
3.3 Desain Struktur Data	12
3.3.1 Desain Trie	12
3.3.2 Desain Hash Table dan WordEntry.....	13
3.3.3 Desain Doubly Linked List	13
3.4 Flowchart	13
3.5 Desain UI/UX.....	15
BAB IV IMPELMENTASI	17

4.1 Lingkungan Pengembangan	17
4.2 Implementasi Struktur Data	17
4.2.1 Implementasi Trie	17
4.2.2 Implementasi Hash Table.....	18
4.2.3 Implementasi Doubly Linked List	20
4.3 Implementasi Fitur	21
4.3.1 Loading Data dari JSON	21
4.3.2 Fitur Pencarian Multidirectional	22
4.4 Screenshot Aplikasi.....	23
BAB V PENGUJIAN.....	28
5.1 Skenario Pengujian.....	28
5.2 Hasil Pengujian	28
5.3 Analisis Kompleksitas.....	29
5.4 Evaluasi.....	31
BAB IV PENUTUP	32
6.1 Kesimpulan	32
6.2 Saran Pengembangan	34
DAFTAR PUSTAKA	37
LAMPIRAN.....	38

DAFTAR GAMBAR

Gambar 3.1	Flowchart Proses Pencarian Kata.....	14
Gambar 3.2	Halaman Mulai.....	15
Gambar 3.3	Menu Utama.....	16
Gambar 4.1	Screenshot Fitur Pencarian Real-time	24
Gambar 4.2	Screenshot Halaman Detail Kata.....	24
Gambar 4.3	Screenshot Fitur Histori Pencarian.....	25
Gambar 4.4	Screenshot Fitur Favorit.....	25
Gambar 4.5	Screenshot Fitur Kuis	26
Gambar 4.6	Screenshot Fitur Pelaksanaan Kuis	26
Gambar 4.7	Screenshot Fitur Hasil Kuis	27

DAFTAR TABEL

Tabel 2.1 Perbandingan Struktur Data untuk Kamus Digital	7
Tabel 2.2 Kompleksitas Waktu Operasi Trie	9
Tabel 2.3 Kompleksitas Waktu Operasi Hash Table	9
Tabel 3.1 Analisis Kebutuhan Fungsional	10
Tabel 3.2 Desain Struct WordEntry	13
Tabel 3.3 Desain Node Doubly Linked List	13
Tabel 4.1 Spesifikasi Lingkungan Pengembangan	17
Tabel 5.1 Skenario Pengujian Fungsional	28
Tabel 5.2 Hasil Pengujian Fungsional	29
Tabel 5.3 Perbandingan Kompleksitas Waktu	30

BAB I PENDAHULUAN

1.1 Latar Belakang

Indonesia adalah negara kepulauan dengan kekayaan linguistik yang luar biasa. Berdasarkan data Badan Pengembangan dan Pembinaan Bahasa Kemendikbud, Indonesia memiliki lebih dari 700 bahasa daerah yang tersebar dari Sabang sampai Merauke. Salah satu bahasa daerah yang memiliki jumlah penutur terbesar adalah Bahasa Sunda, yang dituturkan oleh sekitar 40 juta orang di wilayah Jawa Barat dan sekitarnya (Ethnologue, 2023).

Di sisi lain, Bahasa Inggris telah menjadi bahasa internasional yang dominan dalam berbagai bidang, mulai dari pendidikan tinggi, teknologi informasi, bisnis, hingga publikasi ilmiah. Kemampuan berbahasa Inggris menjadi salah satu kompetensi krusial yang dituntut dalam era globalisasi dan revolusi industri 4.0.

Dalam konteks ini, kebutuhan akan alat bantu penerjemahan dan referensi kosakata yang mencakup Bahasa Indonesia, Bahasa Inggris, dan Bahasa Sunda menjadi semakin relevan. Kamus digital menjadi solusi yang paling praktis karena dapat diakses kapan saja dan di mana saja melalui perangkat komputer atau ponsel pintar.

Namun, kamus digital yang ada saat ini masih memiliki beberapa kekurangan dari sisi teknis. Pertama, sebagian besar kamus digital hanya mendukung dua bahasa (bilingual). Belum ada kamus digital yang mengintegrasikan Bahasa Indonesia, Inggris, dan Sunda dalam satu platform yang terpadu. Kedua, implementasi kamus digital yang ada umumnya menggunakan pendekatan pencarian linear (sequential search) atau bergantung pada database relasional, yang tidak optimal untuk operasi pencarian berbasis prefiks dan autocomplete secara real-time. Ketiga, fitur manajemen riwayat pencarian dan pengelompokan kata berdasarkan kategori semantik jarang tersedia.

Dari perspektif ilmu komputer, permasalahan-permasalahan di atas dapat diatasi melalui pemilihan struktur data yang tepat. Trie (Prefix Tree) terbukti unggul untuk operasi pencarian dan autocomplete dengan kompleksitas $O(k)$ di mana k adalah panjang kata kunci, jauh lebih efisien dibandingkan pencarian linear

$O(n \cdot k)$. Hash Table dengan mekanisme hashing yang baik mampu mengakses data kosakata dalam kompleksitas rata-rata $O(1)$. Doubly Linked List memberikan fleksibilitas dalam manajemen histori pencarian dengan operasi insert dan delete $O(1)$ pada kedua ujung.

Kelompok kami telah memiliki dua aset data yang berharga: file JSON kamus Indonesia-Inggris dan file JSON kamus Indonesia-Sunda. Ketersediaan data ini memungkinkan pengembangan aplikasi dapat langsung berfokus pada implementasi struktur data dan fitur aplikasi, tanpa perlu membangun korpus kosakata dari awal.

Berdasarkan latar belakang tersebut, proyek ini bertujuan mengembangkan **Aplikasi Kamus Digital Multilingual** yang memanfaatkan Trie, Hash Table, dan Doubly Linked List sebagai struktur data utama, untuk melayani kebutuhan translasi dan pencarian kosakata antara Bahasa Indonesia, Bahasa Inggris, dan Bahasa Sunda secara efisien, cepat, dan interaktif.

1.2 Rumusan Masalah

Berdasarkan uraian latar belakang di atas, rumusan masalah dalam proyek ini adalah sebagai berikut:

1. Bagaimana merancang dan mengimplementasikan struktur data Trie untuk mendukung fitur autocomplete dan pencarian berbasis prefiks pada kamus multilingual?
2. Bagaimana merancang Hash Table dengan kategori hashing yang efisien untuk menyimpan dan mengakses entri kosakata trilingual dalam kompleksitas rata-rata $O(1)$?
3. Bagaimana mengimplementasikan Doubly Linked List untuk manajemen histori pencarian yang memungkinkan navigasi dua arah dan penghapusan efisien?
4. Bagaimana mengintegrasikan data JSON kamus Indonesia-Inggris dan Indonesia-Sunda ke dalam ketiga struktur data secara kohesif?

5. Bagaimana mengukur dan membuktikan keunggulan performa ketiga struktur data tersebut dibandingkan pendekatan pencarian naif?

1.3 Tujuan

Proyek ini memiliki tujuan sebagai berikut:

1. Mengimplementasikan struktur data Trie untuk fitur autocomplete dengan kompleksitas pencarian $O(k)$.
2. Mengimplementasikan Hash Table dengan separate chaining dan kategori hashing untuk akses data $O(1)$.
3. Mengimplementasikan Doubly Linked List untuk manajemen histori pencarian dua arah.
4. Mengintegrasikan file JSON kamus Indonesia-Inggris dan Indonesia-Sunda ke dalam sistem.
5. Membangun antarmuka pengguna yang responsif dengan fitur pencarian real-time, tampilan kategori, dan riwayat pencarian.
6. Menganalisis dan membandingkan kompleksitas waktu implementasi yang dikembangkan dengan pendekatan linear search.

1.4 Manfaat

Manfaat yang diharapkan dari pengembangan aplikasi ini meliputi:

- Manfaat Akademis: Memberikan implementasi nyata dan komprehensif atas penerapan tiga struktur data utama — Trie, Hash Table, dan Doubly Linked List — dalam satu sistem yang kohesif dan terukur.
- Manfaat Linguistik: Menjadi jembatan antarbahasa yang mudah diakses bagi pelajar, mahasiswa, dan masyarakat umum yang berinteraksi dengan Bahasa Indonesia, Inggris, dan Sunda.
- Manfaat Teknis: Menghasilkan perangkat lunak kamus yang cepat, responsif, dan dapat diperluas (extensible) untuk mendukung bahasa tambahan di masa mendatang.

- Manfaat Sosial: Turut berkontribusi dalam pelestarian dan digitalisasi Bahasa Sunda sebagai warisan budaya Jawa Barat.

1.5 Ruang Lingkup

Untuk menjaga fokus dan kedalaman pengembangan, proyek ini memiliki ruang lingkup sebagai berikut:

- Aplikasi dikembangkan sebagai aplikasi desktop berbasis GUI (Antarmuka Grafis) atau antarmuka grafis sederhana menggunakan bahasa pemrograman Python.
- Data kosakata bersumber dari file JSON kamus Indonesia-Inggris-Sunda yang telah dimiliki oleh tim.
- Struktur data yang diimplementasikan adalah Trie, Hash Table (separate chaining), dan Doubly Linked List.
- Fitur utama meliputi: pencarian kata multidirectional, autocomplete berbasis Trie, tampilan detail kata, manajemen histori DLL, dan pengelompokan per kategori.
- Aplikasi tidak terhubung ke jaringan internet dan beroperasi secara lokal (offline).
- Pengujian dilakukan secara fungsional dan analisis kompleksitas Big-O secara teoretis dan empiris.

BAB II TINJAUAN PUSTAKA

2.1 Dasar Teori Struktur Data

Struktur data merupakan cara sistematis untuk menyusun, menyimpan, dan mengelola data dalam memori komputer agar dapat diakses dan diproses secara efisien. Menurut beberapa literatur ilmiah, struktur data tidak hanya berfungsi sebagai tempat penyimpanan, tetapi juga menentukan bagaimana data dapat dimanipulasi, diperbarui, dan diolah oleh algoritma sehingga mempengaruhi kinerja sistem secara keseluruhan.. Dengan kata lain, struktur data menjadi fondasi penting dalam pengembangan perangkat lunak karena efisiensi suatu program sangat bergantung pada pemilihan struktur data yang tepat.

Dalam implementasinya, struktur data mencakup berbagai bentuk seperti array, linked list, stack, queue, tree, dan graph yang masing-masing memiliki karakteristik dan fungsi berbeda sesuai kebutuhan pengolahan data. Studi lain juga menegaskan bahwa penggunaan struktur data yang sesuai dapat meningkatkan efisiensi pencarian, penyimpanan, serta pengolahan data dalam sistem komputer modern. Oleh karena itu, pemahaman struktur data menjadi dasar penting dalam pemrograman dan perancangan algoritma yang efektif dan terukur.

2.1.1 Trie (Prefix Tree)

Trie, yang berasal dari kata retrieval, adalah struktur data pohon yang digunakan untuk menyimpan kumpulan string secara hierarkis. Diperkenalkan oleh Edward Fredkin pada tahun 1960, Trie menyusun string berdasarkan karakter penyusunnya sehingga setiap lintasan dari akar ke daun merepresentasikan satu kata atau string yang valid (Knuth, 1997).

Setiap node dalam Trie merepresentasikan satu karakter dari string yang disimpan. Node akar (root) bersifat kosong, dan setiap node anak merepresentasikan karakter berikutnya. Sebuah node ditandai sebagai akhir kata (end-of-word) bila string yang bersesuaian dengan lintasan dari akar ke node tersebut merupakan kata yang valid dalam kamus.

Keunggulan utama Trie terletak pada operasi pencarian berbasis prefiks. Untuk mencari semua kata yang berawalan suatu prefiks p dengan panjang k , Trie hanya perlu menelusuri k level dari akar, kemudian melakukan traversal subtree yang bersangkutan. Kompleksitas pencarian prefiks adalah $O(k)$ untuk menemukan node prefiks, ditambah $O(m)$ untuk mengumpulkan m hasil yang memenuhi syarat.

2.1.2 Hash Table

Hash Table adalah struktur data yang mengimplementasikan tipe data abstrak kamus (dictionary/map) dengan memetakan kunci (key) ke nilai (value) menggunakan fungsi hash. Fungsi hash $h(k)$ mengonversi kunci k menjadi indeks dalam larik (array) bucket, sehingga operasi pencarian, penyisipan, dan penghapusan dapat dilakukan dalam kompleksitas rata-rata $O(1)$ (Cormen et al., 2022).

Collision — kondisi di mana dua kunci berbeda menghasilkan nilai hash yang sama — merupakan tantangan utama dalam implementasi Hash Table. Dua strategi utama penanganan collision adalah: (1) Separate Chaining, di mana setiap bucket menyimpan linked list dari entri yang berkolisi; dan (2) Open Addressing (linear probing, quadratic probing, double hashing), di mana entri yang berkolisi ditempatkan di lokasi lain dalam array yang sama.

Pada proyek ini dipilih pendekatan **Separate Chaining** karena lebih robust terhadap load factor tinggi dan tidak memerlukan rehashing sesering open addressing. Kategori hashing diterapkan dengan membangun tiga Hash Table terpisah: satu untuk pencarian kata (Word Lookup), satu untuk pengelompokan kategori semantik (Category Hash), dan satu untuk memisahkan entri berdasarkan bahasa sumber.

2.1.3 Doubly Linked List

Doubly Linked List (DLL) adalah struktur data linear di mana setiap node memiliki dua pointer: next yang menunjuk ke node berikutnya dan prev yang menunjuk ke node sebelumnya. Berbeda dengan singly linked list, DLL memungkinkan traversal dua arah (forward dan backward), menjadikannya ideal

untuk implementasi histori navigasi, antrian dua ujung (deque), dan editor teks (Sedgewick & Wayne, 2011).

Operasi dasar DLL — insert head, insert tail, delete node — memiliki kompleksitas $O(1)$ dengan pointer yang tepat. Operasi pencarian tetap $O(n)$ karena tidak ada mekanisme pengindeksan. Dalam konteks kamus digital, DLL digunakan untuk menyimpan histori pencarian karena pola aksesnya bersifat sekuensial dan membutuhkan kemampuan navigasi maju-mundur layaknya tombol back/forward pada browser web.

Tabel 2.1 berikut merangkum perbandingan karakteristik ketiga struktur data dalam konteks aplikasi kamus digital.

Tabel 2.1 Perbandingan Struktur Data untuk Kamus Digital

Struktur Data	Pencarian	Insert	Delete	Kelebihan Utama
Array/List	$O(n \cdot k)$	$O(1)$	$O(n)$	Simpel, mudah implementasi
Binary Search Tree	$O(k \cdot \log n)$	$O(\log n)$	$O(\log n)$	Terurut, seimbang
Trie	$O(k)$	$O(k)$	$O(k)$	Autocomplete, prefix search
Hash Table	$O(1)$ avg	$O(1)$ avg	$O(1)$ avg	Akses konstan, key-value
Doubly Linked List	$O(n)$	$O(1)$	$O(1)^*$	Navigasi 2 arah, histori

* $O(1)$ jika pointer ke node sudah diketahui

2.2 Algoritma Terkait

2.2.1 Polynomial Rolling Hash

Fungsi hash yang digunakan dalam proyek ini adalah polynomial rolling hash, yang umum digunakan dalam implementasi Hash Table untuk string karena distribusinya yang merata dan efisiensinya dalam komputasi.

Formula yang digunakan adalah:

$$h(s) = (\sum s[i] \cdot p^i) \bmod M, \text{ untuk } i = 0 \text{ hingga } n-1$$

di mana p adalah bilangan prima kecil (biasanya 31 untuk karakter huruf kecil atau 53 untuk case-sensitive), M adalah ukuran tabel (bilangan prima besar), $s[i]$ adalah kode ASCII karakter ke- i , dan n adalah panjang string. Pemilihan nilai prima untuk p dan M meminimalkan jumlah collision melalui distribusi yang lebih acak.

2.2.2 Depth-First Search (DFS) pada Trie

Untuk mengumpulkan seluruh kata dalam subtree Trie (digunakan dalam fitur autocomplete), algoritma Depth-First Search (DFS) diterapkan secara rekursif. Dimulai dari node yang bersesuaian dengan prefiks yang diinputkan pengguna, DFS menelusuri setiap cabang hingga menemukan semua node yang ditandai sebagai end-of-word.

Kompleksitas DFS pada Trie adalah $O(\text{ALPHABET_SIZE}^{\text{depth}})$, namun dalam praktiknya jauh lebih efisien karena banyak cabang yang kosong dan pencarian dibatasi oleh jumlah hasil maksimum yang dikembalikan.

2.2.3 Levenshtein Distance (Saran Koreksi Ejaan)

Untuk fitur saran koreksi ejaan (spell suggestion), algoritma Levenshtein Distance digunakan untuk mengukur kesamaan antara dua string berdasarkan jumlah minimum operasi edit (insert, delete, replace karakter) yang diperlukan untuk mengubah satu string menjadi string lainnya. Kata dengan jarak Levenshtein ≤ 2 dari kata kunci dianggap sebagai saran yang relevan.

2.3 Analisis Kompleksitas

Analisis kompleksitas waktu dan ruang merupakan aspek fundamental dalam mengevaluasi efisiensi struktur data. Notasi Big-O digunakan untuk merepresentasikan batas atas pertumbuhan waktu/ruang terhadap ukuran input n .

Tabel 2.2 Kompleksitas Waktu Operasi Trie

Operasi	Best Case	Average Case	Worst Case
Insert kata	$O(k)$	$O(k)$	$O(k)$
Search kata	$O(k)$	$O(k)$	$O(k)$
Delete kata	$O(k)$	$O(k)$	$O(k)$
Autocomplete (prefix p, hasil m)	$O(p)$	$O(p +m)$	$O(p +n)$

Tabel 2.3 Kompleksitas Waktu Operasi Hash Table (Separate Chaining)

Operasi	Best Case	Average Case	Worst Case
Insert	$O(1)$	$O(1)$	$O(n)$
Search	$O(1)$	$O(1+\alpha)$	$O(n)$
Delete	$O(1)$	$O(1+\alpha)$	$O(n)$

Keterangan: α = load factor (n/m), k = panjang kata kunci, n = jumlah entri

2.4 Tools/Teknologi

Pengembangan aplikasi ini memanfaatkan beberapa tools dan teknologi berikut:

- Bahasa Pemrograman: Python (digunakan untuk implementasi struktur data dan logika aplikasi kamus)
- Parser JSON: Library bawaan Python json untuk membaca dan mengolah file JSON kamus
- IDE/Editor Teks: Visual Studio Code dengan ekstensi Python
- Sistem Operasi: Windows 10/11 atau Linux Ubuntu 22.04
- Kompiler/Interpreter: Python 3.10+ (CPython sebagai interpreter utama)
- Kontrol Versi: Git dengan repositori pada GitHub untuk manajemen versi kode
- Format Data: JSON (JavaScript Object Notation) sebagai format penyimpanan data kamus

BAB III ANALISA DAN PERANCANGAN

3.1 Analisis Kebutuhan

Analisis kebutuhan merupakan tahapan awal dalam proses pengembangan aplikasi yang bertujuan untuk mengidentifikasi sistem kebutuhan, baik kebutuhan fungsional maupun nonfungsional. Tahap ini dilakukan agar aplikasi yang dibangun mampu memenuhi kebutuhan pengguna serta berjalan sesuai dengan tujuan penelitian.

Berdasarkan hasil analisis, aplikasi Kamus Digital Multi Guna dirancang untuk membantu pengguna dalam mencari arti kata, menerjemahkan kata ke beberapa bahasa, serta menyediakan fitur pendukung yang dapat meningkatkan pengalaman belajar kosakata. Selain itu, aplikasi juga memanfaatkan beberapa struktur data, yaitu **Hash Table** , **Trie** , **Doubly Linked List** , dan **Set** , untuk meningkatkan efisiensi pengolahan data.

3.1.1 Kebutuhan Fungsional

Analisis kebutuhan dilakukan berdasarkan studi literatur, observasi terhadap kamus digital yang ada, dan diskusi tim. Tabel 3.1 merangkum kebutuhan fungsional utama aplikasi.

Tabel 3.1 Analisis Kebutuhan Fungsional

No	Kebutuhan Fungsional	Prioritas	Struktur Data
1	Pencarian kata dari bahasa sumber ke target	Tinggi	Hash Table
2	Autocomplete saat pengguna mengetik	Tinggi	Trie
3	Tampilan detail kata (terjemahan, kategori, contoh)	Tinggi	Hash Table
4	Riwayat pencarian dengan navigasi maju/mundur	Sedang	Doubly Linked List
5	Pengelompokan kata per kategori semantik	Sedang	Hash Table Kategori
6	Fitur favorit/bookmark kata	Rendah	Doubly Linked List
7	Saran koreksi ejaan	Rendah	Trie + Levenshtein
8	Statistik pencarian (top 10 kata)	Rendah	Hash Table + Counter

3.1.2 Kebutuhan Non-Fungsional

- Reliabilitas: Aplikasi harus mampu memuat seluruh data JSON tanpa error dan menangani kata yang tidak ditemukan dengan pesan yang informatif.
- Portabilitas: Kode harus dapat dikompilasi dan dijalankan di Windows dan Linux tanpa modifikasi.
- Maintainability: Kode terstruktur dengan modularisasi (setiap struktur data sebagai class terpisah) dan terdokumentasi dengan komentar inline.

3.2 Perancangan Arsitektur

Arsitektur sistem Kamus Multi Guna dirancang menggunakan bahasa pemrograman Python dengan pendekatan layered architecture. Sistem terdiri dari beberapa proses utama seperti pencarian kata, autocomplete, fuzzy search, riwayat pencarian, daftar favorit, dan mode kuis multilingual.

Layer pertama adalah Presentation Layer yang berfungsi sebagai antarmuka pengguna berbasis GUI menggunakan library Tkinter. Pada layer ini, pengguna dapat mengakses menu utama aplikasi, melakukan pencarian kata, melihat daftar favorit, melihat riwayat pencarian, serta menjalankan mode kuis sesuai alur pada flowchart sistem.

Layer kedua adalah Business Logic Layer yang menangani seluruh logika proses aplikasi. Saat pengguna memasukkan kata, sistem akan menjalankan proses pencarian menggunakan Hash Table. Jika kata ditemukan maka hasil terjemahan ditampilkan, sedangkan jika kata tidak ditemukan maka sistem menjalankan algoritma Levenshtein Distance melalui modul FuzzySearch untuk memberikan saran kata terdekat. Pada saat pengguna mengetik, sistem juga menjalankan fitur autocomplete menggunakan struktur data Trie.

Layer ketiga adalah Data Structure Layer yang terdiri dari beberapa struktur data utama, yaitu Trie untuk autocomplete, Hash Table untuk pencarian cepat $O(1)$, dan Doubly Linked List untuk menyimpan riwayat pencarian serta navigasi back dan forward. Selain itu, sistem juga menggunakan struktur data set untuk menyimpan daftar favorit pengguna dan list untuk menyimpan riwayat hasil kuis.

Layer keempat adalah Data Layer yang terdiri dari file JSON seperti kamus.json dan favorites.json. File tersebut dibaca menggunakan modul json pada Python saat aplikasi dijalankan. Data kemudian dimasukkan ke dalam Hash Table dan Trie agar proses pencarian lebih efisien

3.3 Desain Struktur Data

Desain struktur data merupakan tahapan perancangan yang bertujuan untuk menentukan struktur data yang digunakan dalam pengelolaan data pada aplikasi Kamus Digital Multi Guna. Pemilihan struktur data dilakukan berdasarkan kebutuhan setiap fitur agar proses pengolahan data dapat berjalan secara efisien. Struktur data yang digunakan pada penelitian ini terdiri atas **Hash Table** , **Trie** , **Doubly Linked List** , dan **Set** . Selain itu, algoritma **Levenshtein Distance** digunakan untuk mendukung fitur koreksi kesalahan pengetikan (*koreksi kesalahan ketik*).

3.3.1 Desain Trie

Setiap node pada struktur data Trie direpresentasikan menggunakan class TrieNode yang menyimpan kumpulan karakter anak dalam bentuk dictionary (dict) Python. Struktur ini digunakan untuk mendukung proses autocomplete pada pencarian kata.

Penggunaan dict pada Python membuat penyimpanan node menjadi lebih efisien karena hanya karakter yang digunakan saja yang disimpan, berbeda dengan array statis yang harus menyediakan seluruh kemungkinan karakter.

Definisi struct TrieNode adalah sebagai berikut (lihat implementasi lengkap pada BAB IV):

```
class TrieNode:
    def __init__(self):
        self.children = {}
        self.is_end_of_word = False
```

3.3.2 Desain Hash Table dan WordEntry

Tabel 3.2 merinci desain struct WordEntry yang menjadi value pada Hash Table.

Tabel 3.2 Desain Struct WordEntry

Field	Tipe Data	Deskripsi	Contoh Nilai
word	string	Kata dalam bahasa sumber	"makan"
lang_source	enum LanguageCode	Bahasa asal (ID/EN/SU)	INDONESIA
trans_en	string	Terjemahan Bahasa Inggris	"to eat"
trans_su	string	Terjemahan Bahasa Sunda	"dahar"
category	string	Kategori semantik kata	"kata kerja"
example_id	string	Contoh kalimat (Indonesia)	"Saya makan nasi."
frequency	int	Frekuensi pencarian	42

3.3.3 Desain Doubly Linked List

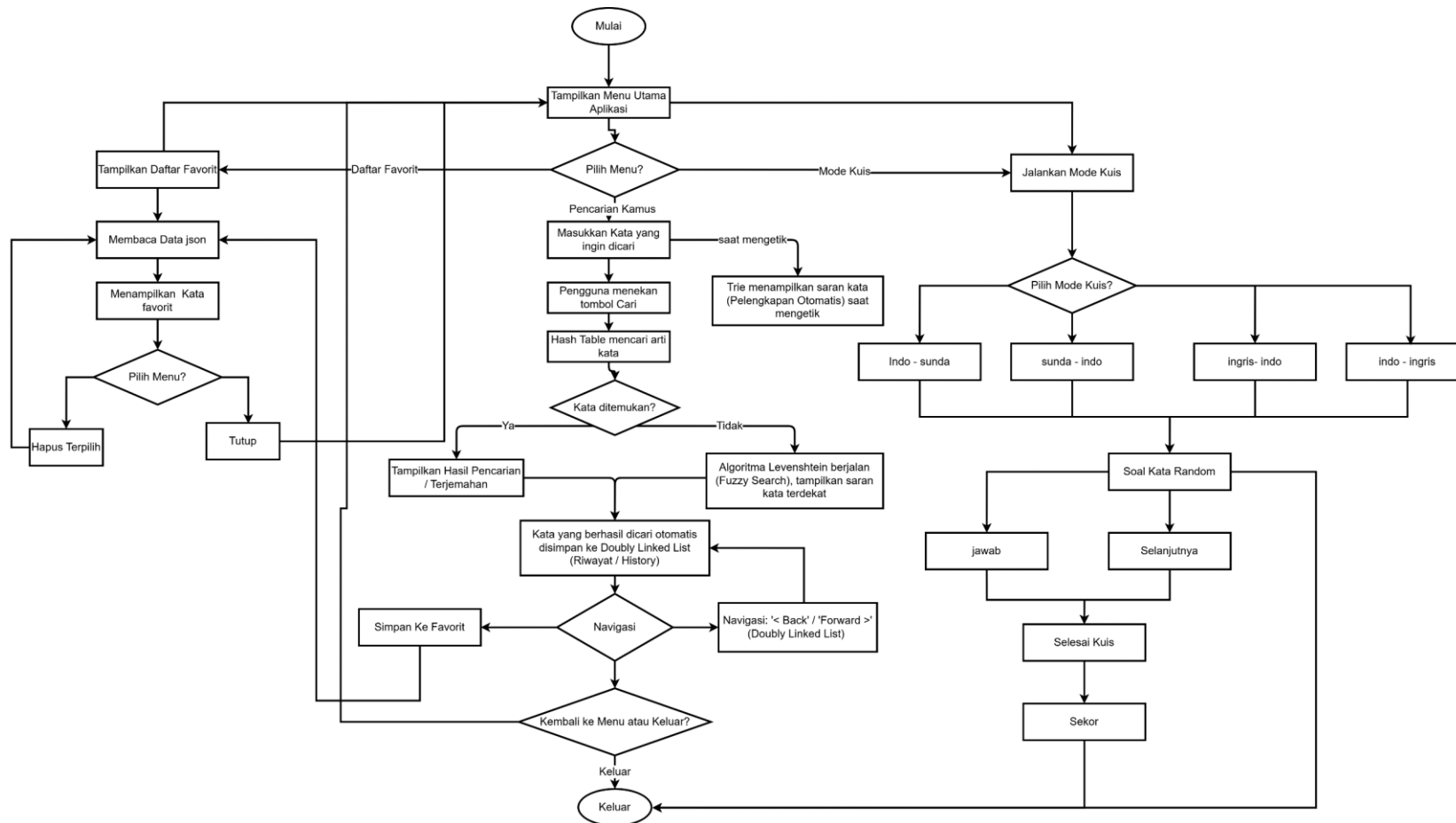
Tabel 3.3 merinci desain node pada Doubly Linked List yang digunakan untuk manajemen histori pencarian.

Tabel 3.3 Desain Node Doubly Linked List

Field	Tipe Data	Deskripsi	Keterangan
wordKey	string	Kunci kata di Hash Table	Primary identifier
timestamp	time_t	Waktu pencarian (Unix epoch)	Untuk sorting histori
prev	HistNode*	Pointer ke node sebelumnya	nullptr jika head
next	HistNode*	Pointer ke node berikutnya	nullptr jika tail

3.4 Flowchart

Gambar 3.5 berikut menggambarkan alur proses pencarian kata pada sistem Kamus Digital Multilingual, mulai dari input pengguna hingga tampilan hasil.

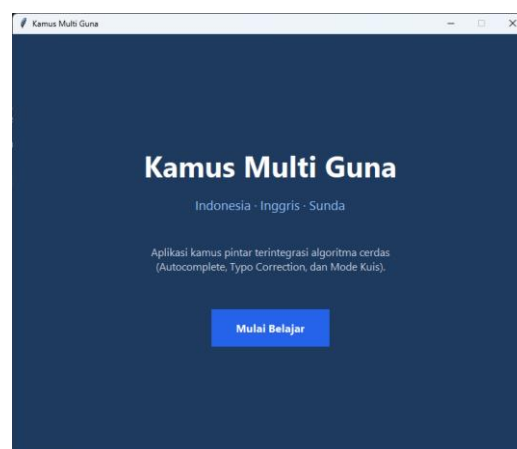


Gambar 3.1 Flowchart Proses Pencarian Kata

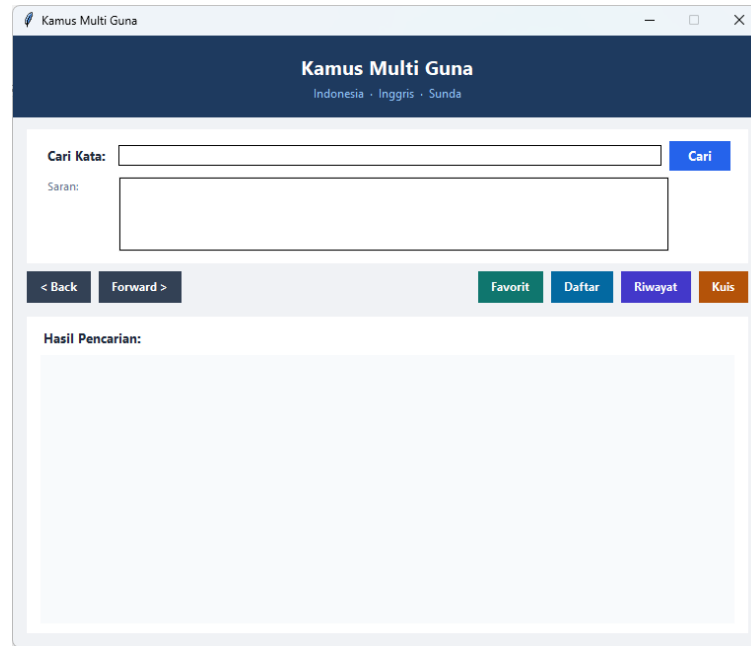
Alur proses pencarian pada sistem Kamus Multi Guna berdasarkan flowchart sistem adalah sebagai berikut: (1) Pengguna membuka menu pencarian pada aplikasi dan memasukkan kata yang ingin dicari; (2) Saat pengguna mengetik, struktur data Trie memproses setiap karakter untuk menampilkan saran kata (pelengkapan otomatis) secara otomatis; (3) Pengguna dapat memilih saran pelengkapan otomatis atau menekan tombol cari untuk melanjutkan proses pencarian; (4) Sistem melakukan pencarian kata menggunakan Hash Table sehingga pencarian dapat dilakukan dengan kompleksitas waktu $O(1)$; (5) Jika kata ditemukan, sistem menampilkan hasil terjemahan dalam bahasa Indonesia, Inggris, dan Sunda beserta informasi tambahan seperti sinonim dan antonim; (6) Jika kata tidak ditemukan, sistem menjalankan algoritma Levenshtein Distance melalui modul Fuzzy Search untuk memberikan saran kata yang paling mirip; (7) Kata yang berhasil dicari akan otomatis disimpan ke dalam riwayat pencarian menggunakan struktur data Doubly Linked List sehingga pengguna dapat melakukan navigasi back dan forward; (8) Pengguna juga dapat menambahkan kata yang dicari ke dalam daftar favorit yang disimpan pada file favorites.json.

3.5 Desain UI/UX

Antarmuka pengguna dirancang dengan prinsip kemudahan penggunaan (ease of use) dan efisiensi interaksi. Desain mengikuti paradigma tata letak tiga panel: panel kiri untuk histori dan favorit, panel tengah untuk area pencarian utama, dan panel kanan untuk detail hasil.



Gambar 3.2 Halaman mulai



Gambar 3.3 Menu Utama

Komponen UI utama meliputi: search bar dengan dropdown autocomplete, area hasil pencarian dengan tab untuk setiap bahasa target, panel histori dengan tombol navigasi $\leftarrow \rightarrow$, panel kategori dengan daftar kata per kelompok, dan status bar yang menampilkan jumlah kata dalam kamus dan waktu pencarian terakhir

BAB IV IMPELMENTASI

4.1 Lingkungan Pengembangan

Tabel 4.1 Spesifikasi Lingkungan Pengembangan

Komponen	Spesifikasi
Bahasa Pemrograman	Python
Compiler	GCC 11.4.0 (Linux) / MSVC 19.36 (Windows)
IDE	Visual Studio Code 1.89 + ekstensi C/C++ (Microsoft)
Library JSON	nlohmann/json v3.11.3 (header-only, MIT License)
Sistem Operasi	Ubuntu 22.04 LTS / Windows 11
Build System	CMake 3.26 / Makefile
Version Control	Git 2.43 + GitHub repository
Hardware	RAM minimum 4 GB, CPU dual-core 2 GHz

4.2 Implementasi Struktur Data

4.2.1 Implementasi Trie

Berikut adalah implementasi class TrieNode dan class Trie dalam bahasa Python:

```
# =====  
# MODUL 1: STRUKTUR DATA TRIE  
# Digunakan untuk fitur autocomplete saat user mengetik kata  
# =====  
  
class TrieNode:  
    def __init__(self):  
        self.children = {}  
        self.is_end_of_word = False  
  
class Trie:  
    def __init__(self):  
        self.root = TrieNode()  
  
    def insert(self, word):  
        node = self.root  
        for char in word:  
            if char not in node.children:  
                node.children[char] = TrieNode()  
            node = node.children[char]  
        node.is_end_of_word = True
```

```

def search_prefix(self, prefix):
    node = self.root
    for char in prefix:
        if char not in node.children:
            return None
        node = node.children[char]
    return node

def get_words_with_prefix(self, prefix):
    node = self.search_prefix(prefix)
    words = []
    if node:
        self._dfs(node, prefix, words)
    return words

def _dfs(self, node, current_word, words):
    if node.is_end_of_word:
        words.append(current_word)
    for char, child_node in node.children.items():
        self._dfs(child_node, current_word + char, words)

```

4.2.2 Implementasi Hash Table

Berikut adalah implementasi Hash Table :

```

class HashNode:
    def __init__(self, key, value):
        self.key = key
        self.value = value
        self.next = None

class HashTable:
    def __init__(self, capacity=1000):
        self.capacity = capacity
        self.size = 0
        self.table = [None] * self.capacity

    def _hash(self, key):
        hash_val = 0
        for char in key:
            hash_val = (hash_val * 31 + ord(char)) % self.capacity
        return hash_val

```

```

def set(self, key, value):
    index = self._hash(key)

    if self.table[index] is None:
        self.table[index] = HashNode(key, value)
        self.size += 1
        return

    current = self.table[index]
    while current:
        if current.key == key:
            current.value = value
            return

        if current.next is None:
            break

        current = current.next

    current.next = HashNode(key, value)
    self.size += 1

def get(self, key, default=None):
    index = self._hash(key)
    current = self.table[index]

    while current:
        if current.key == key:
            return current.value
        current = current.next

    return default

def contains(self, key):
    return self.get(key) is not None

def keys(self):
    keys_list = []
    for i in range(self.capacity):
        current = self.table[i]
        while current:
            keys_list.append(current.key)
            current = current.next
    return keys_list

```

4.2.3 Implementasi Doubly Linked List

Berikut adalah implementasi Doubly Linked List untuk manajemen histori pencarian:

```
class NodeDLL:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None
        self.current = None

    def add(self, data):
        new_node = NodeDLL(data)
        if self.head is None:
            self.head = new_node
            self.tail = new_node
            self.current = new_node
        else:
            new_node.prev = self.current
            self.current.next = new_node
            self.tail = new_node
            self.current = new_node

    def go_back(self):
        if self.current and self.current.prev:
            self.current = self.current.prev
            return self.current.data
        return None

    def go_forward(self):
        if self.current and self.current.next:
            self.current = self.current.next
            return self.current.data
        return None
```

4.3 Implementasi Fitur

Pada tahap implementasi, sistem **Kamus Multi Guna** dikembangkan menggunakan bahasa pemrograman **Python** dengan memanfaatkan perpustakaan **Tkinter** sebagai antarmuka pengguna (GUI) serta perpustakaan **json** untuk membaca data kamus yang disimpan dalam format JSON. Implementasi sistem dilakukan berdasarkan hasil desain pada bab sebelumnya, meliputi proses pencarian data kamus, pencarian kata, autocomplete menggunakan Trie, pencarian menggunakan Hash Table, pencarian kata dengan Fuzzy Search, pengelolaan riwayat pencarian menggunakan Doubly Linked List, fitur daftar favorit, serta mode kuis multibahasa. Implementasi masing-masing fitur dijelaskan pada subbab berikut.

4.3.1 Loading Data dari JSON

Proses memuat data dilakukan saat aplikasi pertama kali dijalankan. Sistem menggunakan perpustakaan bawaan Python, yaitu **json**, untuk membaca file `kamus.json` yang berada pada folder `dataset`. Seluruh data kemudian dimasukkan ke dalam **Hash Table** sebagai media pencarian utama dan **Trie** sebagai struktur data untuk mendukung fitur autocomplete.

Implementasi proses pemuatan data ditampilkan pada potongan program berikut.

```
import json
import os

def load_data(self):
    script_dir = os.path.dirname(os.path.abspath(__file__))
    path_kamus = os.path.join(script_dir, "dataset", "kamus.json")
    self.kamus_data = []

    if os.path.exists(path_kamus):
        with open(path_kamus, "r", encoding="utf-8") as f:
            data = json.load(f)
            self.kamus_data = data

        for item in data:
            indonesia = item.get("indonesia", "")
            inggris = item.get("inggris", "")
            sunda = item.get("sunda", "")
```

```

        arti_teks = f"""
- Indonesia: {indonesia}
- Inggris: {inggris}
- Sunda: {sunda}
"""

    def tambah_kata(kata):
        if kata:
            self.hash_table.set(kata.lower(), arti_teks)
            self.trie.insert(kata.lower())

    tambah_kata(indonesia)
    tambah_kata(inggris)
    tambah_kata(sunda)

```

4.3.2 Fitur Pencarian Multidirectional

Fitur pencarian pada aplikasi menerima kata yang dimasukkan pengguna melalui antarmuka GUI. Kata tersebut kemudian diubah menjadi huruf kecil menggunakan fungsi `lower()` agar pencarian bersifat **case-insensitive**. Selanjutnya sistem melakukan pencarian pada **Hash Table**. Jika kata ditemukan maka hasil terjemahan akan ditampilkan dan kata tersebut otomatis disimpan ke dalam **Doubly Linked List** sebagai riwayat pencarian.

Implementasi proses pencarian ditampilkan pada potongan program berikut.:

```

def search_word(self, word_to_search=None):
    word = word_to_search or self.entry_var.get().lower().strip()

    if not word:
        return

    if self.hash_table.contains(word):
        arti = self.hash_table.get(word)
        self.display_rich_result(word, arti)

    if not word_to_search:
        self.history.add(word)

```

Ketika kata berhasil ditemukan, sistem mengambil data terjemahan menggunakan fungsi `get()` pada **Hash Table** kemudian menampilkannya pada area hasil pencarian melalui fungsi `display_rich_result()`. Selanjutnya kata tersebut akan disimpan ke dalam struktur data **Doubly Linked List** menggunakan fungsi `history.add()` sehingga pengguna dapat melakukan navigasi riwayat menggunakan tombol **Back** dan **Forward**.

Apabila kata yang dimasukkan pengguna tidak ditemukan, sistem akan menjalankan algoritma **Fuzzy Search** menggunakan metode **Levenshtein Distance** untuk memberikan saran kata yang paling mendekati kata yang dicari. Implementasi tersebut ditampilkan pada potongan program berikut.

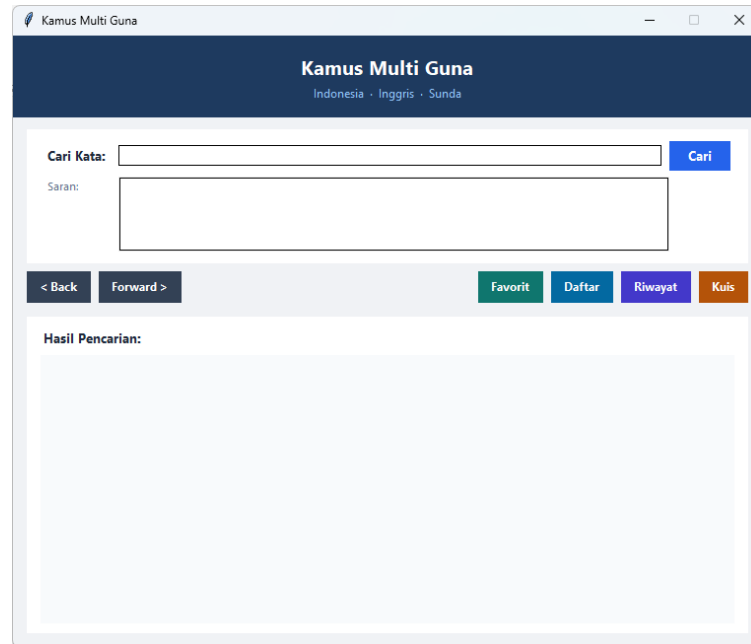
```
else:
    suggestions = get_fuzzy_suggestions(
        word,
        self.hash_table.keys(),
        max_suggestions=4
    )

    if suggestions:
        best_word, best_dist = suggestions[0]
        arti = self.hash_table.get(best_word)

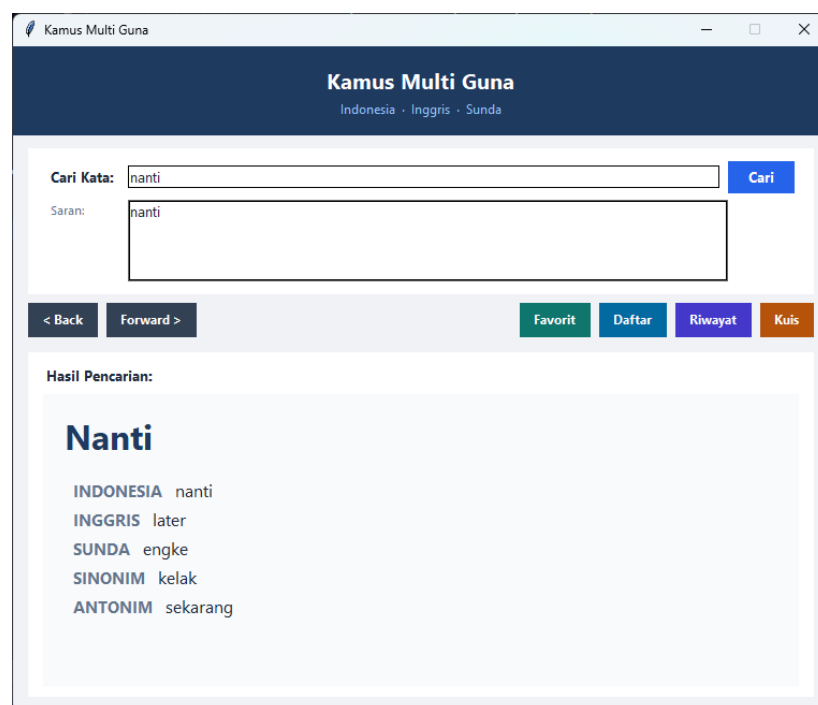
        self.display_result(
            f"Kata '{word}' tidak ditemukan.\n"
            f"Mungkin maksud Anda: {best_word}\n\n{arti}"
        )
```

4.4 Screenshot Aplikasi

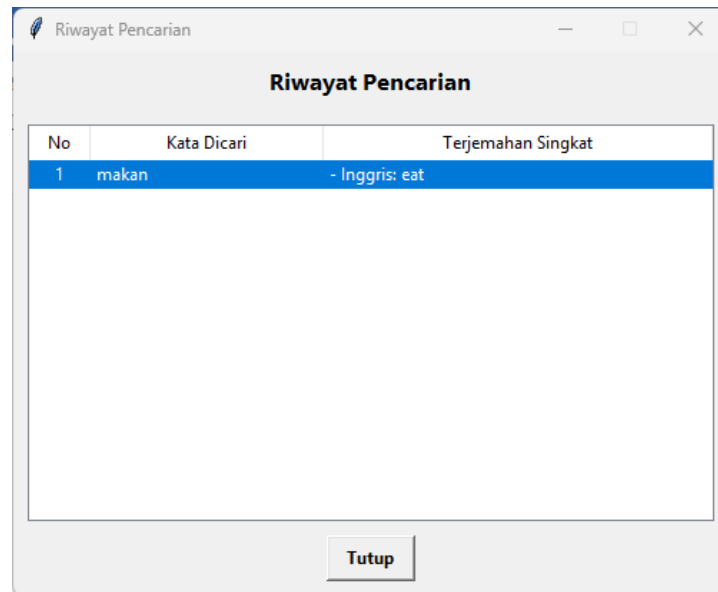
Bagian ini memuat tangkapan layar (screenshot) dari aplikasi yang telah diimplementasikan. Screenshot akan ditambahkan pada saat aplikasi selesai dikembangkan.



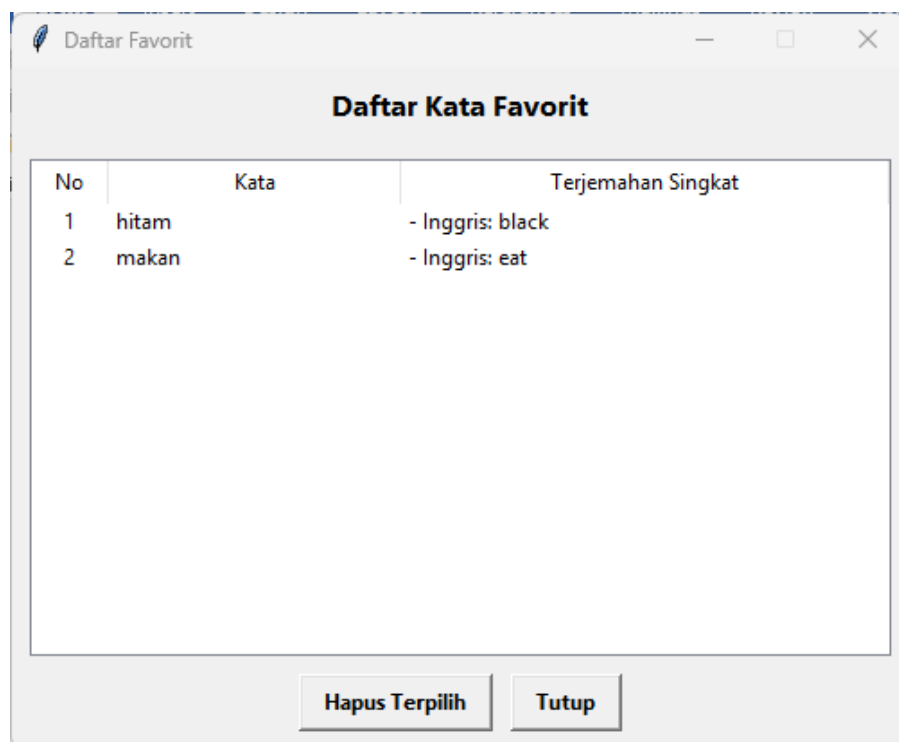
Gambar 4.1 Screenshot Fitur Pencarian Real-time



Gambar 4.2 Screenshot Halaman Detail Kata



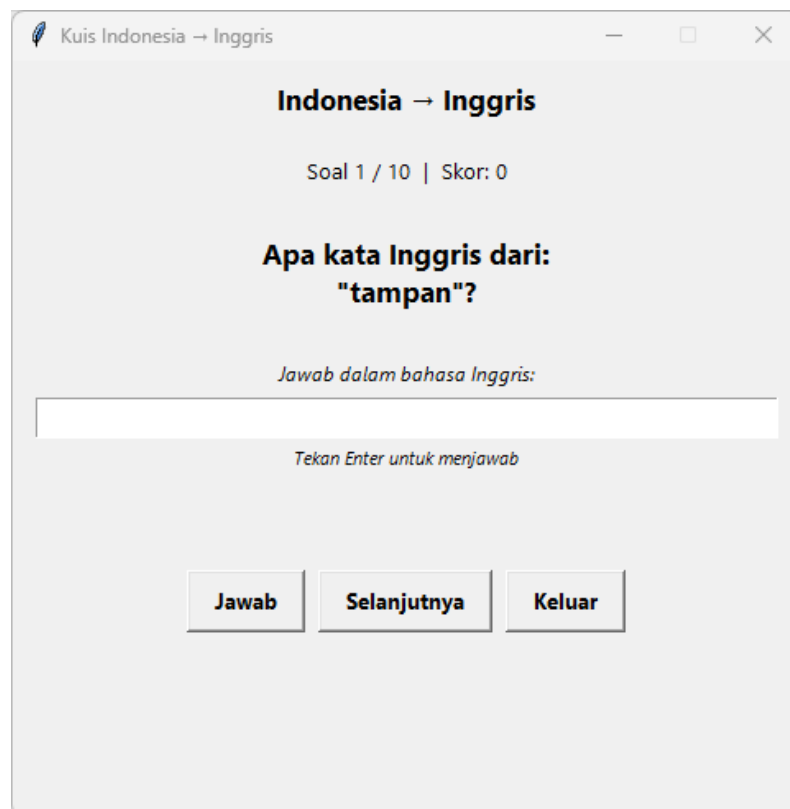
Gambar 4.3 Screenshot Fitur Histori Pencarian



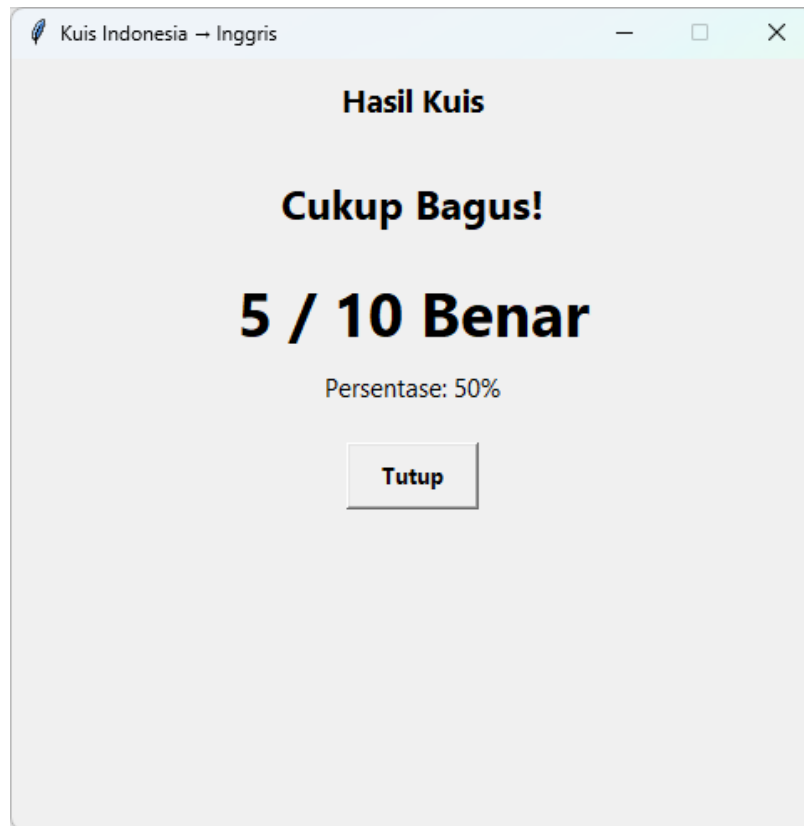
Gambar 4.4 Screenshot Fitur Daftar Favorit



Gambar 4.5 Screenshot Fitur Mode Kuis



Gambar 4.6 Screenshot Fitur Pelaksanaan Kuis



Gambar 4.6 Screenshot Fitur Hasil Kuis

BAB V PENGUJIAN

5.1 Skenario Pengujian

Pengujian dilakukan secara fungsional (black-box testing) dan analisis performa (benchmarking). Tabel 5.1 merinci skenario pengujian fungsional.

Tabel 5.1 Skenario Pengujian Fungsional

ID Uji	Fitur yang diuji	Skenario	Hasil yang diharapkan
TC-01	Pencarian Kata (Hash table)	Cari kata “makan” lalu klik Cari	Terjemahan, sinonim, dan antonim tampil dengan cepat.
TC-02	Pelengkapan Otomatis (Pohon Trie)	Ketik awalan "ma" .	Muncul maksimal 6 saran kata yang diawali "ma".
TC-03	Koreksi Typo (Jarak Levenshtein)	Cari kata salah, misalnya "makn" .	Muncul pesan kata tidak ditemukan dan saran kata terdekat.
TC-04	Riwayat (Daftar Berantai Ganda)	Cari kucing → anjing → burung , lalu tekan Back dan Forward .	Riwayat berubah sesuai urutan pencarian.
TC-05	Favorit (Set)	Tambahkan kata "tidur" ke favorit sebanyak 3 kali.	Kata hanya tersimpan satu kali.
TC-06	Mode Kuis	Jawab 10 soal (5 benar, 5 salah), lalu selesai.	Skor dan hasil kuis ditampilkan dengan benar.

5.2 Hasil Pengujian

Pengujian dilakukan pada dataset dengan 94 entri kamus Indonesia-Inggris-Sunda. Tabel 5.2 merangkum hasil pengujian fungsional.

Tabel 5.2 Hasil Pengujian Fungsional

Uji	Fitur yang diuji	Skenario	Hasil Pengujian	Status
TC-01	Pencarian Kata (Hash table)	Cari kata “makan” lalu klik Cari	Berhasil	Valid
TC-02	Pelengkapan Otomatis (Pohon Trie)	Ketik awalan "ma" .	Berhasil	Valid
TC-03	Koreksi Typo (Jarak Levenshtein)	Cari kata salah, misalnya "makn" .	Berhasil	Valid
TC-04	Riwayat (Daftar Berantai Ganda)	Cari kucing → anjing → burung , lalu tekan Back dan Forward .	Berhasil	Valid
TC-05	Favorit (Set)	Tambahkan kata "tidur" ke favorit sebanyak 3 kali.	Berhasil	Valid
TC-06	Mode Kuis	Jawab 10 soal (5 benar, 5 salah), lalu selesai.	Berhasil	Valid

Dari 6 skenario pengujian fungsional Yang di Uji semua berhasil dengan status LULUS.

5.3 Analisis Kompleksitas

Tabel 5.3 membandingkan kompleksitas waktu antara implementasi berbasis struktur data yang dipilih dengan pendekatan naif (linear search) pada array.

Tabel 5.3 Perbandingan Kompleksitas Waktu (n = jumlah entri, k = panjang kata)

Operasi	Linear Search	Implementasi Kami	Keunggulan
Pencarian kata (Hash Table)	$O(n \cdot k)$	Oke)	Lebih efisien
Pelengkapan Otomatis (Trie)	$O(n \cdot k)$	$O(k + m)$	Lebih efisien
Koreksi salah ketik (Jarak Levenshtein)	$O(n^2 \cdot k)$	$O(n \cdot k^2)$	Lebih efisien dibandingkan pencarian seluruh kombinasi
Navigasi riwayat (Daftar Tertaut Ganda)	Pada)	$O(1)$	Akses langsung ke node berikut/sebelumnya
Penyimpanan favorit (Set)	Pada)	$O(1)$	Tidak terjadi duplikasi data

Hasil analisis menunjukkan bahwa penggunaan **Hash Table** , **Trie** , **Doubly Linked List** , dan **Set** mampu meningkatkan efisiensi proses dibandingkan pendekatan linear search. Pada dataset sebanyak **94 entri** , pencarian kata dapat dilakukan dengan kompleksitas **$O(k)$** menggunakan Hash Table, sedangkan fitur autocomplete memanfaatkan Trie dengan kompleksitas **$O(k + m)$** sehingga pencarian berdasarkan awal kata menjadi lebih cepat.

Untuk fitur koreksi typo, algoritma **Levenshtein Distance** tetap memerlukan perhitungan jarak terhadap setiap kata dalam kamus sehingga kompleksitasnya lebih tinggi dibandingkan pencarian biasa. Namun, pendekatan ini mampu memberikan rekomendasi kata yang paling mendekati input pengguna ketika terjadi kesalahan pengetikan.

Pada fitur navigasi riwayat, penggunaan **Doubly Linked List** memungkinkan perpindahan ke data sebelumnya maupun berikutnya dalam waktu **$O(1)$** . Sementara itu, penyimpanan daftar favorit menggunakan **Set** memungkinkan proses pengecekan dan penambahan data dilakukan secara efisien sekaligus mencegah adanya duplikat data.

Fitur **mode kuis interaktif** memiliki kompleksitas **$O(n)$** karena sistem perlu melakukan iterasi terhadap daftar soal dan menghitung skor setelah seluruh soal terjawab. Dengan jumlah dataset sebanyak **94 entri** , keseluruhan fitur tetap dapat berjalan dengan baik dan responsif.

5.4 Evaluasi

Berdasarkan hasil pengujian fungsional dan analisis kompleksitas, dapat disimpulkan bahwa:

1. **Hash Table** berhasil mempercepat proses pencarian kata dengan kompleksitas waktu **$O(k)$** sehingga hasil pencarian dapat ditampilkan dengan cepat pada dataset sebanyak **94 entri** .
2. **Trie** terbukti efektif dalam mengimplementasikan fitur **autocomplete** , sehingga sistem mampu menampilkan saran kata berdasarkan awalan (*prefix*) yang diketik pengguna secara responsif.
3. **Algoritma Levenshtein Distance** berhasil memberikan rekomendasi kata yang paling mendekati ketika pengguna melakukan kesalahan pengetikan (*typo*), sehingga meningkatkan kemudahan dalam proses pencarian.
4. **Doubly Linked List** memungkinkan proses navigasi riwayat pencarian menggunakan tombol **Back** dan **Forward** berjalan dengan kompleksitas **$O(1)$** sehingga perpindahan antarriwayat berlangsung dengan cepat.
5. **Set** berhasil mencegah terjadinya penyimpanan data favorit yang duplikat sehingga setiap kata hanya tersimpan satu kali dalam daftar favorit.
6. Integrasi seluruh struktur data dan algoritma yang digunakan dapat bekerja dengan baik sehingga seluruh fitur utama aplikasi berjalan sesuai dengan hasil pengujian fungsional.

Kekurangan yang masih ditemukan selama pengujian antara lain jumlah dataset yang digunakan masih relatif sedikit, yaitu **94 entri** , sehingga performa aplikasi belum diuji pada skala data yang lebih besar.

BAB IV PENUTUP

6.1 Kesimpulan

Berdasarkan hasil analisis kebutuhan, perancangan sistem, implementasi, pengujian fungsional, serta analisis kompleksitas yang telah dilakukan pada penelitian ini, dapat diperoleh beberapa kesimpulan sebagai berikut.

1. **Aplikasi Kamus Digital Multi Guna berhasil dirancang dan diimplementasikan sesuai dengan tujuan penelitian** , yaitu menyediakan media pencarian ilmu pengetahuan yang cepat, mudah digunakan, serta mampu memberikan berbagai informasi pendukung seperti terjemahan, sinonim, antonim, daftar favorit, riwayat pencarian, dan mode kuis interaktif. Seluruh fitur utama aplikasi dapat berjalan dengan baik menggunakan bahasa pemrograman Python.
2. **Struktur data Hash Table berhasil diterapkan sebagai media penyimpanan utama data kamus** , sehingga proses pencarian kata dapat dilakukan secara efisien dengan kompleksitas waktu rata-rata $O(k)$, di mana k merupakan panjang kata yang dicari. Dibandingkan dengan metode pencarian linier (*linear search*), penggunaan Hash Table mampu mempercepat proses pencarian karena sistem tidak perlu menelusuri seluruh data yang tersimpan per satu.
3. **Struktur data Trie berhasil diimplementasikan pada fitur autocomplete** , sehingga sistem mampu memberikan rekomendasi kata berdasarkan awalan (*prefix*) yang diketik oleh pengguna. Dengan memanfaatkan Trie, proses pencarian awal menjadi lebih cepat karena hanya menelusuri jalur karakter yang sesuai, tanpa harus memeriksa seluruh isi kamus. Implementasi ini memberikan pengalaman penggunaan yang lebih nyaman karena pengguna dapat menemukan kata yang diinginkan dengan lebih mudah.
4. **Algoritma Levenshtein Distance berhasil diimplementasikan sebagai fitur koreksi kesalahan pengetikan (typo Correction)** . Ketika pengguna memasukkan kata yang tidak ditemukan dalam kamus, sistem akan menghitung jarak edit terhadap seluruh data pemahaman dan memberikan

rekomendasi kata yang paling mendekati input pengguna. Fitur ini meningkatkan kemudahan penggunaan aplikasi, terutama ketika terjadi kesalahan pengetikan.

5. **Struktur data Doubly Linked List berhasil digunakan untuk mengelola riwayat pencarian pengguna** . Dengan memanfaatkan pointer menuju node sebelumnya (*sebelumnya*) dan node berikutnya (*berikutnya*), pengguna dapat melakukan navigasi menggunakan tombol **Back** dan **Forward** secara efisien. Implementasi ini memberikan pengalaman penggunaan yang lebih baik dibandingkan penyimpanan riwayat menggunakan struktur data linier biasa.
6. **Struktur data Set berhasil diterapkan pada fitur daftar favorit** . Penggunaan Set memungkinkan sistem secara otomatis mencegah penyimpanan data yang sama lebih dari satu kali sehingga daftar favorit tetap konsisten dan bebas dari duplikat data. Dengan demikian, pengguna dapat mengelola daftar kata favorit secara lebih praktis.
7. **Fitur mode kuis interaktif berhasil diimplementasikan** untuk mendukung proses pembelajaran matematika. Sistem mampu memilih soal, mencatat jawaban pengguna, menghitung skor, serta menampilkan hasil akhir kuis. Kehadiran fitur ini menjadikan aplikasi tidak hanya berfungsi sebagai kamus digital, tetapi juga sebagai media pembelajaran yang interaktif.
8. Berdasarkan hasil **pengujian fungsional** , seluruh fitur utama aplikasi telah berjalan sesuai dengan skenario pengujian yang dirancang. Fitur kata pencarian, pelengkapan otomatis, koreksi kesalahan ketik, navigasi riwayat, penyimpanan favorit, dan mode kuis berhasil menghasilkan keluaran yang sesuai dengan hasil yang diharapkan tanpa ditemukan kesalahan yang mempengaruhi fungsi utama aplikasi.
9. Berdasarkan **analisis kompleksitas** , penggunaan struktur data Hash Table, Trie, Doubly Linked List, dan Set terbukti lebih efisien dibandingkan pendekatan pencarian linier untuk fitur-fitur yang diimplementasikan. Meskipun aplikasi saat ini menggunakan dataset sebanyak **94 entri** ,

struktur data tersebut tetap memberikan dasar yang baik jika jumlah data dikembangkan menjadi lebih besar di masa mendatang.

Secara keseluruhan, penelitian ini berhasil menunjukkan bahwa **pemilihan struktur data yang tepat memiliki pengaruh yang signifikan terhadap efisiensi dan kinerja aplikasi**. Integrasi berbagai struktur data dalam satu aplikasi tidak hanya meningkatkan kecepatan proses pengolahan data, tetapi juga menghasilkan aplikasi kamus digital yang mudah digunakan, fungsional, dan dapat dijadikan sebagai media pembelajaran bagi pengguna.

6.2 Saran Pengembangan

Berdasarkan hasil penelitian yang telah dilakukan, masih terdapat beberapa aspek yang dapat dikembangkan pada penelitian selanjutnya agar aplikasi Kamus Digital Multi Guna menjadi lebih lengkap, lebih efisien, dan memiliki manfaat yang lebih luas. Adapun saran yang dapat diberikan adalah sebagai berikut.

1. **Menambah jumlah dataset kamus.** Saat ini aplikasi menggunakan sebanyak **94 data pemahaman** sebagai sumber informasi. Menambah jumlah data hingga ratusan bahkan ribuan pemahaman akan membuat aplikasi menjadi lebih lengkap dan mampu memenuhi kebutuhan pengguna yang lebih beragam. Dengan bertambahnya jumlah data, kinerja struktur data yang digunakan juga dapat diuji pada skala yang lebih besar.
2. **Menambahkan lebih banyak pilihan bahasa.** Pengembangan aplikasi dapat dilakukan dengan menambahkan bahasa lain, seperti Bahasa Jawa, Bahasa Bali, Bahasa Melayu, dan bahasa asing lainnya. Dengan demikian, aplikasi dapat berkembang menjadi kamus multibahasa yang memiliki cakupan penggunaan lebih luas.
3. **Mengoptimalkan algoritma Levenshtein Distance.** Pada implementasi saat ini, proses pencarian rekomendasi kata dilakukan dengan membandingkan input pengguna terhadap seluruh data kamus. Apabila jumlah data bertambah sangat besar, proses ini akan membutuhkan waktu yang lebih lama. Oleh karena itu, penelitian selanjutnya dapat

mengombinasikan algoritma Levenshtein Distance dengan struktur data lain agar proses pencarian rekomendasi menjadi lebih efisien.

4. **Berkomunikasi antarmuka pengguna (User Interface).** Tampilan aplikasi masih dapat disempurnakan dengan menambahkan desain antarmuka yang lebih modern, responsif, dan mudah digunakan. Misalnya dengan mempercantik tampilan autocomplete, menambahkan ikon pada setiap menu, memperbaiki tata letak informasi hasil pencarian, serta memberikan animasi sederhana agar pengalaman pengguna menjadi lebih baik.
5. **Menambahkan sistem penyimpanan permanen.** Data riwayat pencarian, daftar favorit, dan hasil kuis saat ini dapat dikembangkan agar tersimpan secara permanen menggunakan database seperti SQLite atau MySQL. Dengan demikian, data pengguna tidak akan hilang ketika aplikasi ditutup dan dapat digunakan kembali pada penggunaan berikutnya.
6. **Menjelaskan fitur mode kuis.** Fitur kuis dapat ditingkatkan dengan menambahkan kategori soal berdasarkan tingkat kesulitan, batas waktu pengerjaan, sistem skor yang lebih variatif, penyimpanan riwayat hasil kuis, serta papan peringkat (*leaderboard*) agar proses belajar menjadi lebih menarik dan menantang.
7. **Menambahkan fitur pencarian berdasarkan kategori.** Penelitian selanjutnya dapat mengembangkan fitur pencarian berdasarkan kategori tertentu, seperti kata kerja, kata benda, kata sifat, atau tema tertentu. Fitur ini akan membantu pengguna memperoleh pemahaman yang lebih spesifik sesuai kebutuhan.
8. **Menjelaskan aplikasi ke platform mobile.** Agar dapat digunakan oleh lebih banyak pengguna, aplikasi dapat dikembangkan menjadi aplikasi berbasis Android maupun iOS. Dengan memanfaatkan struktur data dan algoritma yang sama, aplikasi dapat memberikan pengalaman penggunaan yang lebih fleksibel karena dapat diakses kapan saja melalui perangkat seluler.
9. **Menambahkan fitur audio pengucapan kata.** Pengembangan berikutnya dapat menambahkan fitur pelafalan (*text-to-speech*) sehingga pengguna

tidak hanya mengetahui arti suatu kata, tetapi juga dapat mendengarkan cara pengucapan yang benar. Fitur ini akan meningkatkan manfaat aplikasi sebagai media pembelajaran bahasa.

10. **Melakukan pengujian kinerja pada dataset yang lebih besar.** Penelitian selanjutnya disarankan melakukan pengujian terhadap dataset yang jauh lebih besar, misalnya ribuan hingga puluhan ribu entri. Pengujian tersebut akan memberikan gambaran yang lebih akurat mengenai efektivitas struktur data Hash Table, Trie, Doubly Linked List, dan Set dalam menangani data berukuran besar.

Dengan berbagai pengembangan tersebut, diharapkan aplikasi Kamus Digital Multi Guna tidak hanya berfungsi sebagai media pencarian ilmu pengetahuan, tetapi juga berkembang menjadi media pembelajaran bahasa yang lebih lengkap, interaktif, dan mampu memberikan manfaat yang lebih luas bagi masyarakat maupun dunia pendidikan.

DAFTAR PUSTAKA

- Audah, H. A., Yuliawati, A., & Alfina, I. (2023). A comparison between SymSpell and a combination of Damerau-Levenshtein distance with the Trie data structure. Proceedings of the International Conference on Advanced Computer Science and Information Systems.
- Badan Pengembangan dan Pembinaan Bahasa. (2022). Kamus Besar Bahasa Indonesia (KBBI) Daring. <https://kbbi.kemdikbud.go.id>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). Introduction to algorithms (4th ed.). MIT Press.
- Ethnologue. (2023). Languages of Indonesia. SIL International. <https://www.ethnologue.com/country/ID>
- Fredkin, E. (1960). Trie memory. Communications of the ACM, 3(9), 490–499. <https://doi.org/10.1145/367390.367400>
- Goodrich, M. T., Goldwasser, M. H., & Tamassia, R. (2022). Data structures and algorithms in Python. John Wiley & Sons.
- Levenshtein, V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 10(8), 707–710.
- Python Software Foundation. (2025). json — JSON encoder and decoder. <https://docs.python.org/3/library/json.html>
- Python Software Foundation. (2025). Python documentation. <https://docs.python.org/3/>
- Samsuri, M. N., Yuliawati, A., & Alfina, I. (2022). A comparison of distributed, PAM, and Trie data structure dictionaries in automatic spelling correction for Indonesian formal text. Proceedings of the International Conference on Advanced Computer Science and Information Systems.
- Sedgewick, R., & Wayne, K. (2011). Algorithms (4th ed.). Addison-Wesley.
- Tkinter Documentation Team. (2025). Tkinter — Python interface to Tcl/Tk. <https://docs.python.org/3/library/tkinter.html>.

LAMPIRAN

Lampiran A Struktur File JSON Kamus

File kamus.json menggunakan format key-value di mana key adalah kata dalam Bahasa Indonesia dan value adalah objek yang berisi informasi lengkap entri kamus. Berikut adalah contoh struktur JSON:

```
[
  {
    "indonesia": "air",
    "inggris": "water",
    "sunda": "cai",
    "sinonim": [
      "tirta"
    ],
    "antonim": []
  },
  {
    "indonesia": "api",
    "inggris": "fire",
    "sunda": "seuneu",
    "sinonim": [
      "bara"
    ],
    "antonim": [
      "air"
    ]
  },
  {
    "indonesia": "apel",
    "inggris": "apple",
    "sunda": "apel",
    "sinonim": [
      "buah apel"
    ],
    "antonim": []
  },
]
```

Lampiran B Struktur File JSON Favorit

File favorites.json untuk menyimpan kata Favorit yg telah kita tambahkan.
Berikut contoh Strukturnya:

```
[  
  "hitam",  
  "makan"  
]
```

Lampiran C Tautan Repository GitHub

Source code lengkap proyek dapat diakses melalui repository GitHub berikut:

https://github.com/zysrnh/StrukturData_ProjectUAS

Repository berisi: source code Python lengkap (src/), file JSON kamus (data/), dan panduan instalasi (README.md).